

Welcome back! If you found this question useful,
don't forget to vote both the question and the answers up.

[close this message](#)

Primitive root and discrete logarithm

Asked 12 years, 8 months ago Modified 12 years, 8 months ago Viewed 6k times

Started studying Diffie Hellman key exchange protocol as part of the course. I dont have much maths knowledge. Can somebody explain in simple terms what is the significance of taking
2 Primitive root in DH algorithm.

Also could someone explain about discrete logarithm

 [group-theory](#) [cryptography](#)



[Share](#) [Cite](#) [Edit](#) [Follow](#) [Flag](#)

asked Jul 20, 2011 at 1:59



[user5507](#)

348 **4** **17**

2  Have you tried wikipedia? – [davidlowryduda](#) ♦ Jul 20, 2011 at 2:01

2 Answers

Sorted by: [Highest score \(default\)](#) 

 **10** Let p be a prime. We say that g is a *primitive root* of p (or sometimes modulo p), if the powers $g^1, g^2, g^3, \dots, g^{p-1}$ are congruent, in some order, to $1, 2, 3, \dots, p-1$ (modulo p). Or in simpler terms, when we consider the remainders when g^k is divided by p , all numbers between 1 and $p-1$ are remainders (0 can't be).

 Note that by Fermat's Theorem, $g^{p-1} \equiv 1 \pmod{p}$, so after g^{p-1} , the powers of g start all over again modulo p , so $g^p \equiv g$, $g^{p+1} \equiv g^2$, and so on.

 If you have seen some group theory, we could alternately say that g is a primitive root of p if g is a *generator* of the multiplicative group on non-zero objects modulo p .

 It can be proved using elementary tools that every prime has a primitive root. The proof is not all that easy. Large primes p have many primitive roots.

Here is a small example. Let $p = 7$, and take $g = 2$. The powers of 2, reduced modulo 7, are 2, 4, 1, 2, 4, $\dots$. So we don't get everything, 2 is not a primitive root of 7. Now take $g = 3$. The powers of 3, reduced modulo 7, are 3, 2, 6, 4, 5, 1, 3, $\dots$, so we do get everything, 3 is a primitive root of 7.

Welcome back! If you found this question useful,
don't forget to vote both the question and the answers up.

[close this message](#)

(some people use $0 \leq k \leq p - 2$ instead, I prefer that, it doesn't matter much).

The number k used to be called the *index* of a with respect to the primitive root g . More recently, and universally in Computer Science, the k is called the **discrete logarithm** of a (with respect to the primitive root g).

These "discrete logarithms" have formal properties much like ordinary logarithms. Note in particular that discrete logarithms are *exponents*, just like ordinary logarithms.

What they are good for: Here I will take a glance at the reason for their use in cryptography. If you know g and k , it is **computationally easy** to calculate the remainder when g^k is divided by p , even when p is a huge prime. We use the *Binary Method for Exponentiation* (look this up) or a relative.

But given g and a , it seems to be computationally **very hard** to find the k between 1 and $p - 1$ such that $g^k \equiv a \pmod{p}$. In other words, finding the discrete logarithm when a very large prime is involved, seems to be computationally very difficult.

That makes exponentiation modulo p a useful "trap-door" function, easy to do, but hard to undo, kind of like multiplication of two primes (easy) and factorization of a product of two large primes (seemingly very difficult).

Hope this gives some overview of discrete logarithm. I really cannot add anything about Diffie Hellman details. For one thing, there is a family of DH procedures.

To sum up, the main reason for the usefulness of discrete logarithm is nice algebraic properties, together with the "trap-door" effect.

[Share](#) [Cite](#) [Edit](#) [Follow](#) [Flag](#)

edited Jul 20, 2011 at 19:52

answered Jul 20, 2011 at 2:38



Andrés E. Caicedo

79.2k 10 222 351



André Nicolas

507k 47 563 997



Here is an overview about the Diffie-Hellman key exchange algorithm.

1

See André's answer about the basics of the discrete logarithm.



We have a (cyclic) group (the multiplicative group modulo a big prime is the original case, but other groups are possible, too), in which we can multiply, and thus also have exponentiation with integer numbers (using the square-and-multiply method, for example).



The group G and a generator $g \in G$ (e.g. a primitive root in the prime group case) are fixed before the algorithm starts. These can be public, there is nothing private in this.

Welcome back! If you found this question useful,
don't forget to vote both the question and the answers up.

[close this message](#)

B sends β to A.

3. User A calculates $\gamma_A := \beta^x$.

User B calculates $\gamma_B := \alpha^y$.

Now we have $\gamma_B = \alpha^y = (g^x)^y = g^{x \cdot y} = g^{y \cdot x} = (g^y)^x = \beta^x = \gamma_A$ (without ever calculating $x \cdot y$).

Thus $\gamma := \gamma_A = \gamma_B$ can be used as a shared secret between A and B, to derive a key from it.

But a possible adversary only could have observed α and β , neither x nor y nor γ . The [Diffie-Hellman problem](#) is the problem to compute γ from α and β . Of course, this could be solved easily by calculating the discrete logarithm of α or β , but this is considered *hard*.

There is no proof yet that the Diffie-Hellman problem really can only be solved by calculating discrete logarithms, and it is not yet proved that discrete logarithms are really hard (this also depends on the group in question), but there is no known efficient algorithm, so the Diffie-Hellman key exchange is actually used (for example, in the TLS/SSL and SSH protocols).

In practice you also want to add some authentication to be sure that you are really communicating with the partner you think you are communicating with - the plain DH KE is susceptible to a man-in-the-middle attack (the attacker negotiates one key with A, another one with B, and then decrypts/encrypts data between both).

The [ElGamal encryption scheme](#) is derived from the DH key exchange algorithm - here A publishes α (as well as G and g) ahead of time, and B then chooses y for each message to A, and sends β as well as $m \cdot \gamma$.

Alice can then compute $\gamma^{-1} = \beta^{-x} = \beta^{q-x}$, where q is the order of the group G , and from that the message m .

(In reality, we will only encrypt a key for some symmetric algorithm, and use this to encrypt the real message.)

[Share](#) [Cite](#) [Edit](#) [Follow](#) [Flag](#)

edited Jul 20, 2011 at 20:49

answered Jul 20, 2011 at 20:19



Paulo Ebermann

494 4 16